

VaporPkg: Blocking AI-Hallucinated and Suspicious Package Installs at the Command Line

Varshith

Blekinge Institute of Technology (BTH), Sweden

October 2026

Abstract

Code-generating large language models (LLMs) frequently recommend software packages that do not exist. Because public registries such as PyPI and npm let anyone claim an unused name, an attacker who registers a frequently hallucinated name can deliver malware to every developer who copies the suggested install command, an attack now called *slopsquatting*. This report presents VaporPkg, a dependency-free command-line guard that intercepts `pip`, `uv`, `poetry`, `npm`, `pnpm`, `yarn`, `bun` and `npmx`-style commands, extracts every package they would fetch, and scores each one with explainable signals: existence, registry quarantine status, malicious-package advisories, package age, release history, lookalike similarity to popular names, install-time scripts and provenance metadata. On live registries, VaporPkg raised no alerts on 600 sampled popular packages and warned on 6–9 % of long-tail packages, which are mostly new or single-release projects. It also ships a reproducible benchmark that measures hallucination rate and *persistence* of hallucinated names across models.

1 Introduction

LLM coding assistants are now part of everyday development. When asked for code that needs a third-party library, they often answer with an install command such as `pip install <name>`. Spracklen et al. [1] generated 576,000 code samples with 16 models and found that 19.7 % of recommended packages did not exist (5.2 % for commercial models, 21.7 % for open models), with over 200,000 unique hallucinated names. Many of those names recurred when the same prompt was repeated, which makes them predictable. Security researchers have demonstrated the attack in practice by registering a hallucinated name (`huggingface-cli`) as an empty package and observing thousands of real downloads [2].

Classic typosquatting defences [3] assume the victim mistypes a *popular* name. Hallucinated names are different: they are often plausible compositions (`flask-jwt-simple-auth`) far from any single popular package, so edit-distance checks alone miss them. The most reliable signal is also the simplest one: *the name does not exist yet*, or it was registered very recently.

Contributions.

1. A guard that sits in front of 13 package-manager entry points and refuses to execute installs of non-existent, quarantined, malicious or high-risk packages, with transparent per-signal reasons.
2. A parser that extracts install candidates from shell commands, dependency files (requirements files, `pyproject.toml`, `package.json`) and free text such as LLM answers, including `import/require` statements with `import-to-distribution` mapping.
3. A reproducible benchmark harness for package hallucination (rate, persistence, cross-model overlap) and an alert-rate study on real registry data.

2 Threat model

The attacker can register any unclaimed name on PyPI or npm, publish arbitrary code (including npm install scripts or Python build hooks that run at install time), and choose names that LLMs are likely to produce. The attacker cannot modify the registries themselves or existing packages owned by others (account takeover of popular packages is out of scope, but known malicious versions are surfaced via OSV). The victim is a developer or CI pipeline that installs packages suggested by an LLM without verifying them. VaporPkg’s goal is to stop the install *before* any attacker code runs, at negligible cost for legitimate, established packages.

3 Design

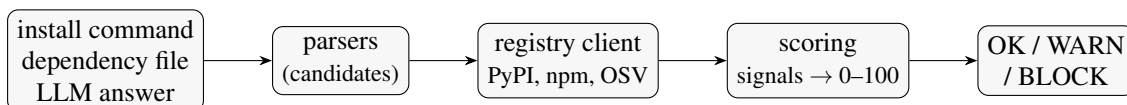


Figure 1: VaporPkg pipeline. Lookups run in parallel and are cached (24 h; 1 h for missing names).

3.1 Candidate extraction

Install commands are tokenised with shell-aware splitting that stops at control operators (&&, ;, |) and comments, and handles chained commands on one line, IPython magics (%pip) and runners (npx, uvx, pipx run), where only the executed package is relevant. Requirement strings follow PEP 508; direct URL references, local paths, wheels and VCS URLs are skipped because they are not resolved from the public index. npm aliases ("x": "npm:real@1") resolve to the real name; GitHub shorthands and file:/link: specifiers are skipped. Candidates found only via import statements are marked low-confidence: standard-library modules (sys.stdlib_module_names) and Node built-ins are removed, and about 90 common import→distribution mismatches (cv2→opencv-python) are mapped.

3.2 Registry evidence

For PyPI the JSON API provides releases with upload times, project URLs and ownership (including organization accounts). When the JSON API returns 404, the Simple API (PEP 691) distinguishes three cases: the name is unregistered, it is registered but has no installable files, or it carries a PEP 792 status such as quarantined. For npm the full packument provides creation time, versions, repository, maintainers and install scripts; popular packages are fetched with the abbreviated install metadata to save bandwidth, in which case age and provenance signals are not computed. npm security placeholders (0.0.1-security) are detected. Optionally, OSV is queried for OpenSSF MAL-* advisories, and weekly download counts are fetched for non-popular packages.

3.3 Scoring

Each signal contributes points (Table 1); critical signals block outright. The default policy blocks at ≥ 70 and warns at ≥ 30 . Packages in the top-15,000 download lists are capped at 20 points unless a critical signal fires, because popular packages legitimately have install scripts or few maintainers. Missing names derived only from imports produce a warning instead of a block.

Signal	Points	Rationale
NOT_FOUND, QUARANTINED, SECURITY_HOLDING, KNOWN_MALICIOUS	block	Hallucinated or removed-for-malware names.
VERY_NEW (<14 d) / NEW (<90 d)	45 / 25	Squats are registered shortly before exploitation.
LOOKALIKE	35 / 20	$\leq 1-2$ edits (optimal string alignment) from a top-5,000 name; separator confusion; popular name plus affix.
SINGLE_RELEASE / FEW_RELEASES	15 / 8	No maintenance history.
VERY_LOW / LOW_DOWNLOADS	20 / 10	< 50 / $< 1,000$ weekly downloads.
INSTALL_SCRIPTS	15	npm pre/post-install hooks execute code.
NO_REPOSITORY / NO_DESCRIPTION	12 / 8	Weak provenance.
DEPRECATED	10	Abandoned or archived.
VERIFIED_ORG	-10	PyPI organization ownership.

Table 1: Risk signals and default weights.

4 Evaluation

4.1 Correctness

An offline test suite (30 tests, fake registry transport) covers PEP 508 edge cases, nested requirement files with hashes and line continuations, `pyproject.toml` (PEP 621, PEP 735 dependency groups, Poetry), `package.json` aliases, pip/npm option parsing, extraction from a realistic LLM answer, all verdict paths, caching, SARIF output and the guard’s refusal to execute blocked installs.

4.2 Alert rates on real packages

To quantify the cost of the policy for legitimate users, VaporPkg was run against live registries on 2026-10-05 (seed 7) for four cohorts (Table 2). The OSV and download-count endpoints were unreachable from the measurement environment, so those two signals were disabled. All other signals were computed from live data.

Cohort	<i>n</i>	OK %	WARN %	BLOCK %	Median lookup
Popular PyPI (top-15k sample)	300	100.0	0.0	0.0	30 ms
Popular npm (top-15k sample)	300	99.7*	0.0	0.0	57 ms
Random PyPI (outside top 15k)	300	91.3	6.0	2.7	29 ms
Long-tail npm (outside top 15k)	300	90.7	9.3	0.0	90 ms

Table 2: Verdict distribution on existing packages. *One transient network error (now retried).

Popular packages produced no alerts. In the uniform random PyPI sample, every BLOCK (8 of 300) was a registered name with *no installable files* (deleted or placeholder projects). Installing those fails anyway, and whoever controls the name could publish code under it later. Warnings in the long tail were dominated by NEW, SINGLE_RELEASE and NO_REPOSITORY. These are not necessarily false positives, since unvetted new packages are exactly the risk class, but they show the friction of the policy. That friction is why WARN prompts the user instead of blocking. Lookalike warnings on short names motivated restricting edit-distance targets to names of at least five characters.

4.3 Package-hallucination benchmark (protocol)

The harness issues 70 coding tasks (40 Python, 30 JavaScript) to any OpenAI-compatible endpoint, by default with three repetitions at temperature 0.7, and measures for each model and language: (i) mention-level and unique-name hallucination rate, (ii) share of answers with at least one hallucinated package, (iii)

persistence, meaning the fraction of (task, hallucinated name) pairs that recur in at least two repetitions, (iv) suggested names that exist but are younger than 90 days (possible squats already registered), (v) cross-model overlap, and (vi) the fraction of hallucinated names VaporPkg blocks. By construction (vi) is 100 % for names that do not exist; the interesting quantities are (i)–(v).

Model	Unique-name rate	Answer rate	Persistence	Recent existing
<i>Fill in from <code>bench/results/summary.md</code> after running the benchmark.</i>				

Table 3: Hallucination benchmark results (to be produced with `run_bench.py + evaluate.py`).

5 Limitations and ethics

VaporPkg raises the cost of slopsquatting but is not a malware scanner: a patient attacker can age a package, add a repository link and inflate downloads. It should complement content scanners and lockfile-based workflows. Lookalike detection depends on snapshot popularity lists, and private indexes are not queried. For ethical reasons, raw lists of hallucinated names produced by the benchmark should not be published. They should instead be reported to the registry security teams so the names can be reserved or monitored.

6 Conclusion

A small amount of registry metadata, evaluated at the moment of installation, stops the most direct form of slopsquatting (installing a name that does not exist or was just registered) with no measurable impact on popular dependencies and a modest, explainable warning rate on the long tail.

References

- [1] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, B. Viswanath, and M. Jadliwala. “We Have a Package for You! A Comprehensive Analysis of Package Hallucinations by Code Generating LLMs.” *USENIX Security Symposium*, 2025. arXiv:2406.10279.
- [2] B. Lanyado. Research on AI package hallucinations and the `huggingface-cli` experiment. Lasso Security, 2024.
- [3] M. Taylor, R. Vaidya, D. Davidson, L. De Carli, and V. Rastogi. “Defending Against Package Typosquatting.” *Network and System Security (NSS)*, 2020.
- [4] M. Ohm, H. Plate, A. Sykosch, and M. Meier. “Backstabber’s Knife Collection: A Review of Open Source Software Supply Chain Attacks.” *DIMVA*, 2020.
- [5] PEP 792 – Project status markers in the simple index. Python Software Foundation, 2025.
- [6] OpenSSF Malicious Packages repository. <https://github.com/ossf/malicious-packages>.